From outer products to competitive GEMM

Understanding SME, diagnosing a 32×32 baseline, and tuning for Apple M4 Pro.

A matrix instruction is powerful. A fast matrix product also needs the right data movement.

01 Understand the hardware interface
02 Find the cost beyond the multiply
03 Tune, verify, and compare honestly

1.67×

over original SME32
at N=4096, total

≈ Accelerate

at N=4096
1.58 TFLOP/s, total

A real single-thread FP32 benchmark, with reproducible data and source.

SME is an instruction set; Accelerate is a library

The comparison is between implementations using a common GEMM problem.

SME: Scalable Matrix Extension

Arm instruction-set extension
Streaming execution mode
Z vectors and predicates
ZA matrix accumulators

Apple Accelerate

A framework with BLAS and LAPACK
cblas_sgemm computes C = A × B
Library chooses its internal path
One BLAS thread in this benchmark

SME2 is an extension of SME; the optimized code uses SME2 multi-vector loads.

We measure complete implementations, including their different packing strategies.

Sources: Arm ACLE [1] · Apple Accelerate [3] · src/accelerate.c

One outer product updates many output values

Matrix multiplication is a sum of outer products: C += A[:, k] × B[k, :].



On this M4 Pro: 16 FP32 lanes × 16 lanes = 256 output updates per FMOPA.

256 fused multiply-adds = 512 nominal FLOPs per FP32 FMOPA.

Sources: Arm ACLE [1] · Hello SME microbenchmarks [2] · numerical example constructed for this deck

ZA holds four logical FP32 tiles on this M4 Pro

Scalable means the register dimensions depend on the streaming vector length (SVL).

A Z vector: 512 bits = 16 FP32 values

SVL / 32 = 16 lanes
ZA storage = 64 × 64 bytes
4 × (16 × 16 × 4) = 4096 bytes

The ISA can scale; this kernel is deliberately specialized for 512-bit SVL.

ZA0.S

16 × 16
256 FP32

ZA1.S

16 × 16
256 FP32

ZA2.S

16 × 16
256 FP32

ZA3.S

16 × 16
256 FP32

Four independent tile accumulators let the kernel interleave its updates.

Sources: Arm ACLE [1] · Hello SME [2] · src/runtime.c · examples/m4-pro/environment.json

Streaming mode surrounds the complete kernel

C attributes describe the streaming interface and ownership of ZA.

Normal caller

Row-major A, B
C = A × B

→

Streaming function body

Load Z vectors → FMOPA into ZA
Store completed tile slices to C

→

Return

Completed C
Caller continues

```
__arm_locally_streaming __arm_new("za")
void gemm(...) {
    // Tile helpers share ZA: __arm_streaming __arm_inout("za")
    // For each output tile: reset/load ZA, loop over K, store C.
}
```

Zero per new output tile; reload C only when accumulating another K block.

Source: Arm ACLE [1] · src/sme32.c · src/sme_opt.c · src/sme_pack.c

The 32×32 baseline already uses all four ZA tiles

At each k: load two A vectors, load two B vectors, issue four FMOPA instructions.

One 32×32 output tile

ZA0.S A0 × B0	**ZA1.S** A0 × B1
ZA2.S A1 × B0	**ZA3.S** A1 × B1

Original data path

Row-major A → pack A16
Row-major B → pack B16
32×32 microkernel, K block = 256
Repeated K blocks accumulate into C

The baseline has strong arithmetic throughput; packing and C traffic still matter.

Source: src/sme32.c · src/pack.c · src/gemm.h

Compare the same operation and timing boundary

M4 Pro · 48 GB · macOS 15.7.3 · Apple Clang 17 · 120 jobs / 840 samples

TOTAL / main comparison

Required packing + full product
Accelerate receives original inputs
Allocation and verification excluded

COMPUTE / diagnostic

Inputs already prepared
All arithmetic and C traffic included
SME32 versus optimized SME only

Throughput = $2N^3$ / seconds
Speedup = baseline time / backend time

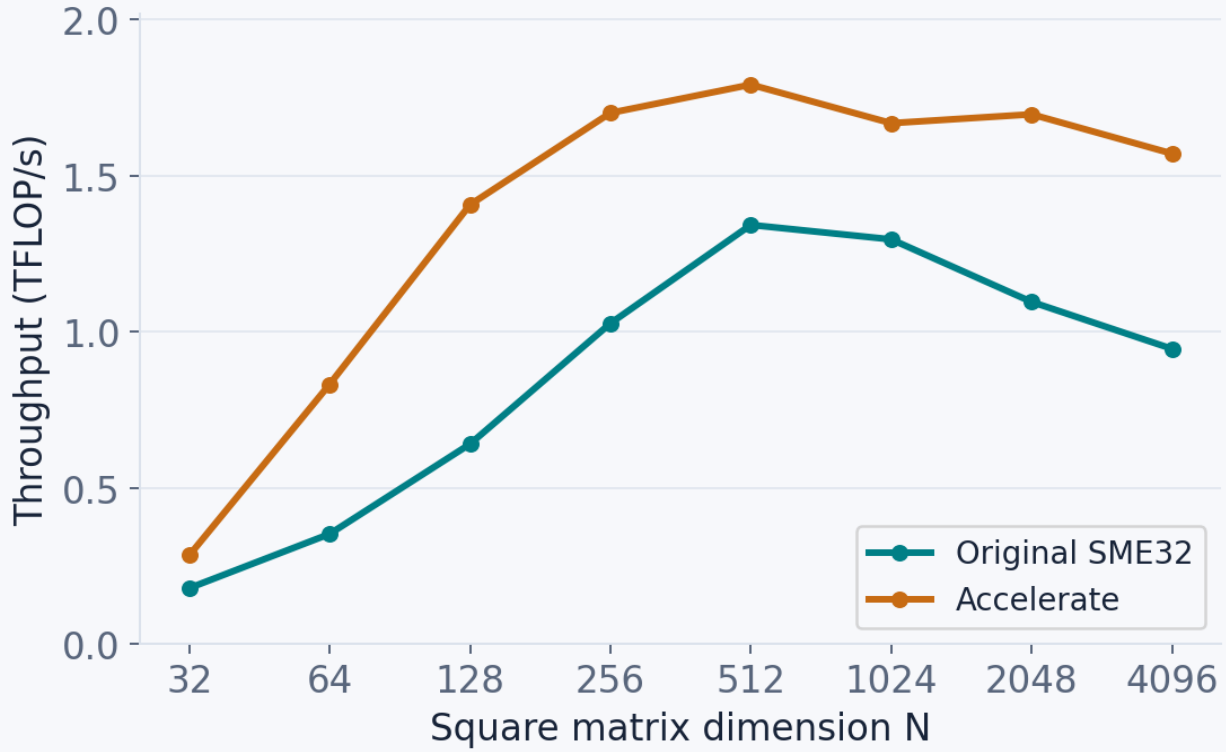
3 rounds × 7 batch-average samples
Shading = paired-round IQR, not CI
Above 1× means faster than baseline

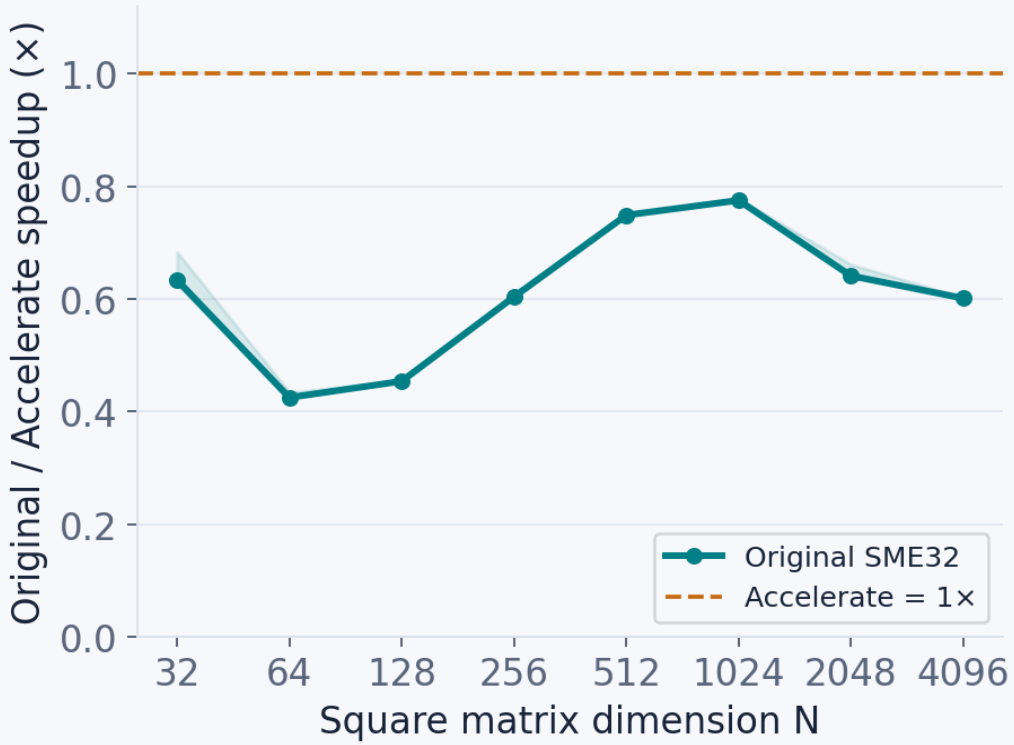
The comparison includes packing and algorithm differences; it does not isolate the ISA.

Sources: main.c · benchmark.py · examples/m4-pro/environment.json, status.json, verification.csv

Original SME32 trails Accelerate end to end

TOTAL mode: both backends start from the same row-major matrices.

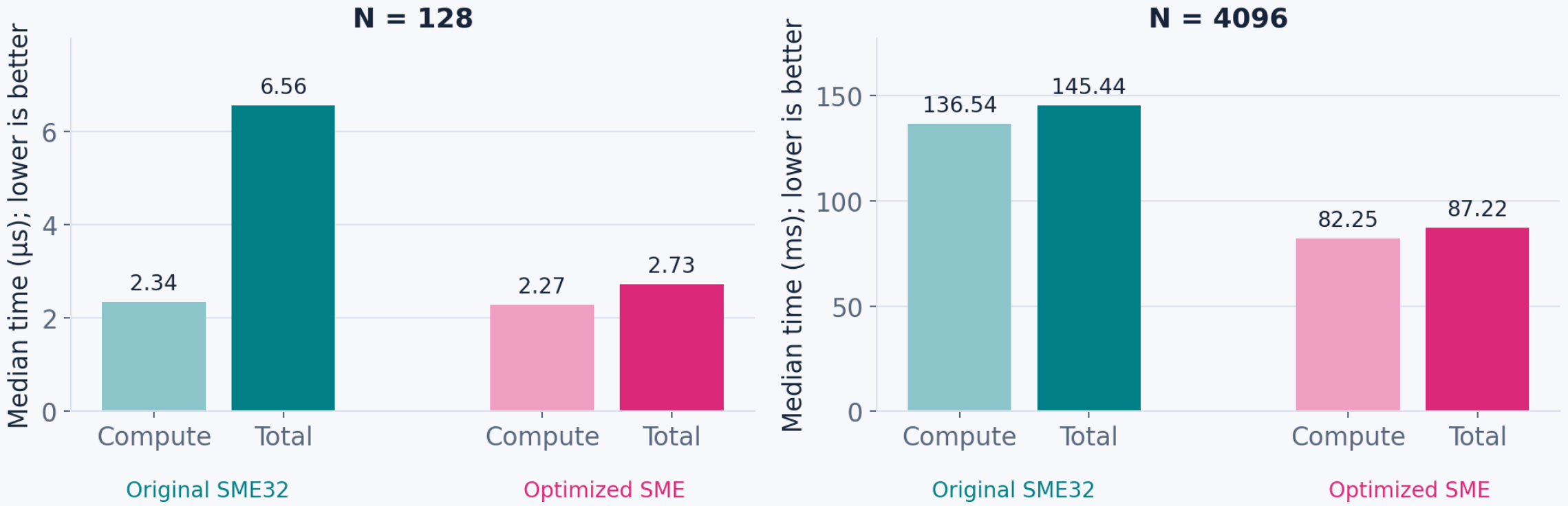




At N=4096: 0.945 vs 1.571 TFLOP/s, including packing.

Small and large matrices reveal different costs

Compute and total are separate measurements; these bars are not additive components.



Small N: reduce preparation cost. Large N: improve the kernel and data reuse.

Measured: examples/m4-pro/ · square FP32 · one calling thread · packing included unless stated

Use a size-specific path with an explicit fallback

The optimized backend chooses its path before entering the GEMM streaming body.

N ≤ 512

Transpose A in ZA
Leave B row-major
32×32 output tiles
No B packing

N > 512, N % 64 = 0

Pack A16 and B64
16×64 output tiles
SME2 grouped loads
Cache blocking

Other supported N

Use original packing
Use original SME32
No padding
No Accelerate call

Supported: square FP32, N a multiple of 32. Benchmark range: N=32…4096.

Each optimization changes the cost of delivering the same mathematical product.

Source: src/sme_opt.c · src/pack_opt.c · src/gemm.h

Transpose A inside ZA to build contiguous panels

Load horizontal slices, read vertical groups, then store packed columns.

Input A: row-major

00	01	02	03
10	11	12	13
20	21	22	23
30	31	32	33

ZA: read columns

00	01	02	03
10	11	12	13
20	21	22	23
30	31	32	33

Packed A: k-major panels

00	10	20	30
01	11	21	31
02	12	22	32
03	13	23	33

Actual block: 16 horizontal loads → 4 vertical-group reads → 4 grouped stores.

For every k, A[k*16 + lane] now supplies 16 consecutive output-row values.

Source: src/sme_pack.c · src/pack.c · intrinsic semantics: Arm ACLE [1]

For small matrices, avoid packing B entirely

At N≤512, adjacent B columns are already contiguous in the original row-major input.

Original

A → pack A16
B → pack B16 → compute

Small optimized path

A → transpose in ZA
B → direct row-major loads → compute

N = 64

3.60× total

1.05× compute only

N = 128

2.40× total

1.03× compute only

N = 512

1.34× total

1.07× compute only

The small-size gain comes mostly from preparing less data, more efficiently.

Source: src/sme_opt.c: direct_b · src/pack_opt.c: pack_inputs_opt · saved paired speedups

Reshape the large-matrix kernel to 16×64

One A vector feeds four FMOPA instructions; B arrives as a four-vector group.



Both shapes: 2048 FLOPs/k. Operand loads: 256 B/k (32×32) vs 320 B/k (16×64).

The gain comes from the complete load/layout/blocking design, not fewer operand bytes.

Source: src/sme_opt.c: tile_16x64 · src/pack_opt.c · FP32/SVL=512 arithmetic

Keep the loaded tuple local to each K iteration

Generated instructions matter: inspect the compiler output as well as the C source.

```
#pragma clang loop unroll_count(4)
for (size_t k = 0; k < kc; ++k) {
  svfloat32_t va = svld1_f32(pg, a+k*16);
  svfloat32x4_t vb = svld1_f32_x4(pg4, b+k*64);
  svmopa_za32_f32_m(0, pg, pg, va, svget4_f32(vb,0));
  svmopa_za32_f32_m(1, pg, pg, va, svget4_f32(vb,1));
  svmopa_za32_f32_m(2, pg, pg, va, svget4_f32(vb,2));
  svmopa_za32_f32_m(3, pg, pg, va, svget4_f32(vb,3));
}
```

Final loop

Load tuple
Consume four B vectors
End tuple lifetime
Repeat, unrolled ×4

Tuning observation: a tuple carried across iterations generated 5 extra SSVE copies/k.

Keep live ranges short and check the actual disassembly on the compiler you use.

Source: src/sme_opt.c: tile_16x64 comment and loop · build.py · build/sme_opt.disasm.txt

Use larger K blocks and bounded M/N traversal

Final compute blocks: KC=2048, MC=256, NC=512. Microkernel: 16×64.

A panel

MC × KC
256 × 2048 FP32
2 MiB logical footprint
16-row micro-panels

B panel

KC × NC
2048 × 512 FP32
4 MiB logical footprint
64-column micro-panels

C at N=4096

Original: 16 K passes
Optimized: 2 K passes
Fewer partial-C updates
Reload only after pass 1

Loop order: K block → N block → M block → 64 columns → 16 rows.

Larger K blocks reduce partial-C traffic; M/N blocks organize operand reuse.

Source: src/gemm.h · src/sme_opt.c: optimized · block sizes are measured tuning choices

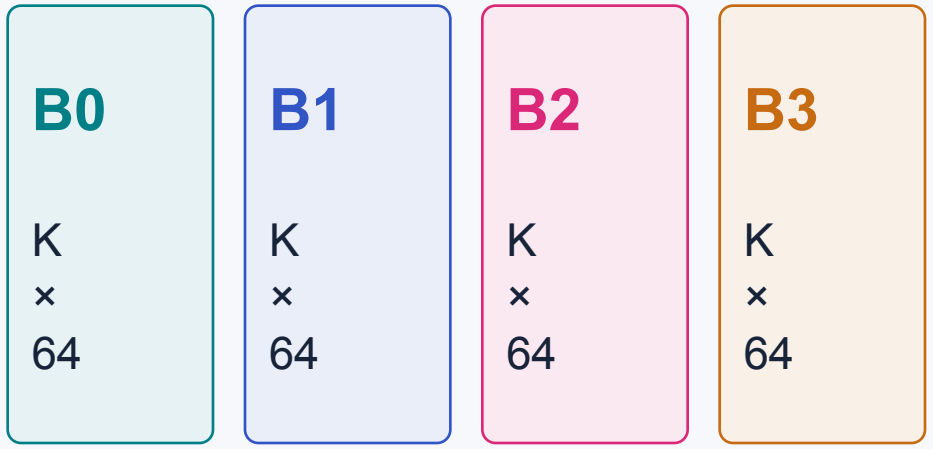
Tile B packing so source and destination stay local

Large B is copied in 32-row × 256-column chunks, then stored as K×64 panels.

Source: original row-major B

A chunk = 32 K rows × 256 columns
32 KiB of logical FP32 source values

Destination: four K×64 panels



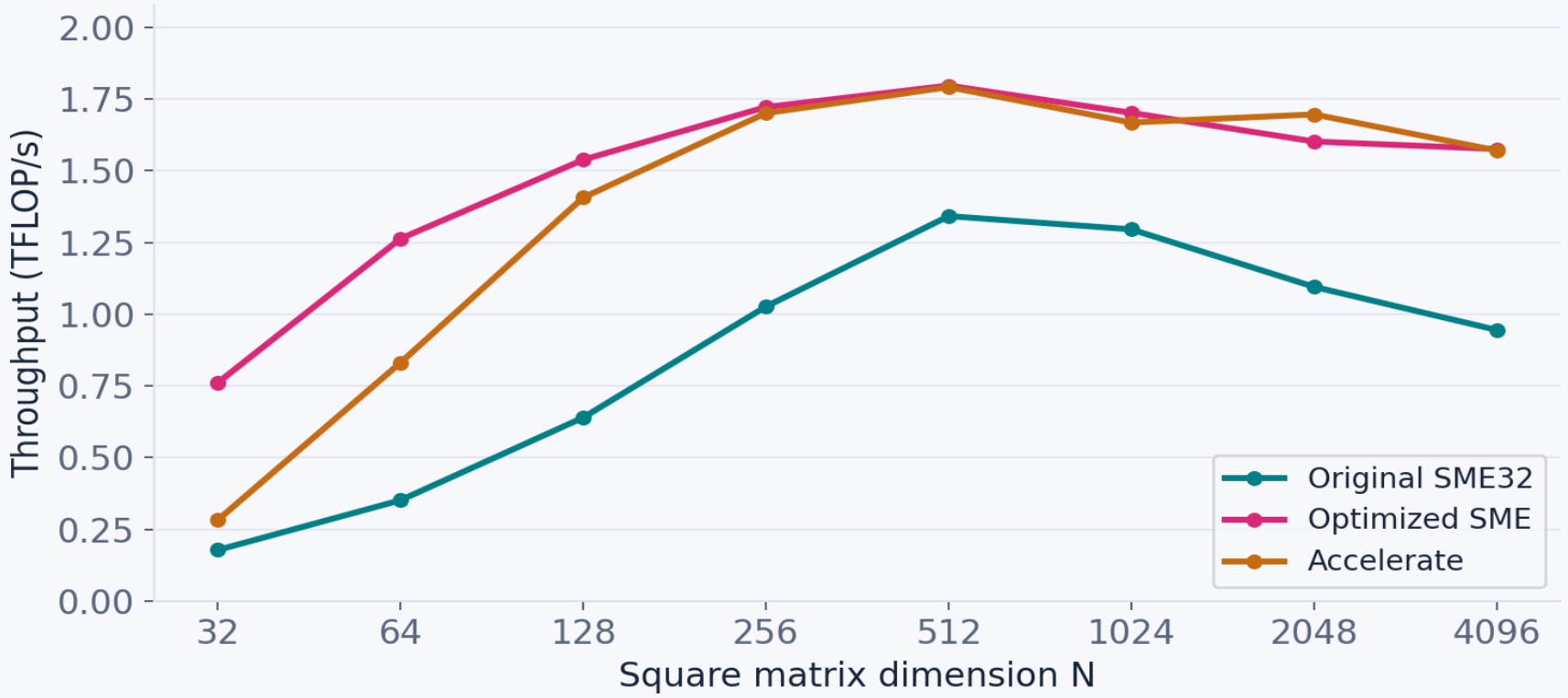
Compute loads 64 consecutive floats
per k using one vector-group load.

Do not trade a faster compute loop for expensive scattered writes in the packer.

Source: src/pack_opt.c: pack_inputs_opt · src/gemm.h

The optimized version closes most of the library gap

TOTAL mode, including required packing. Higher throughput is better.



N = 512

Optimized
1.80 TFLOP/s

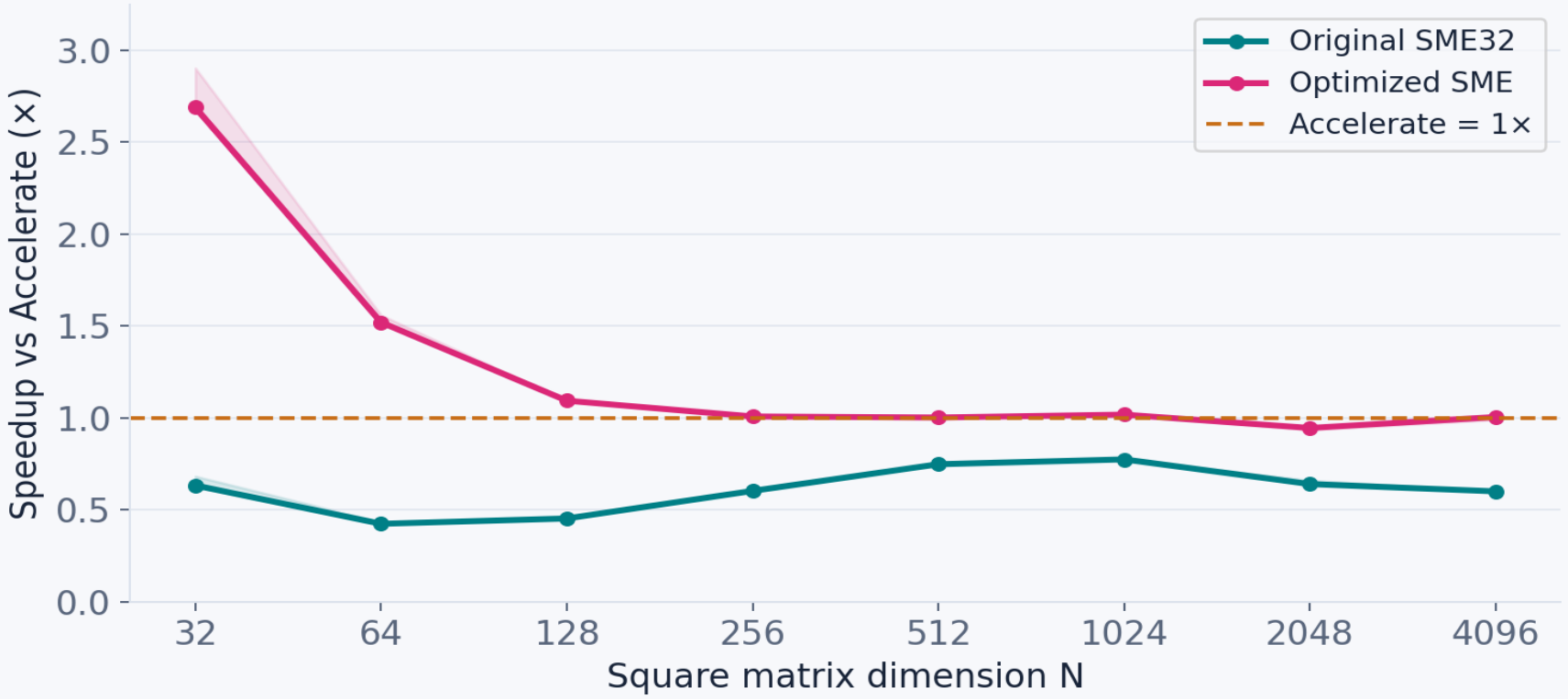
N = 4096

Optimized
1.58 TFLOP/s

Near Accelerate at many sizes; N=2048 remains a visible gap.

Against Accelerate: small-size wins, large-size parity

Speedup = Accelerate time / backend time. Above 1× means faster than Accelerate.



N = 32

2.69×
small-call
benefit

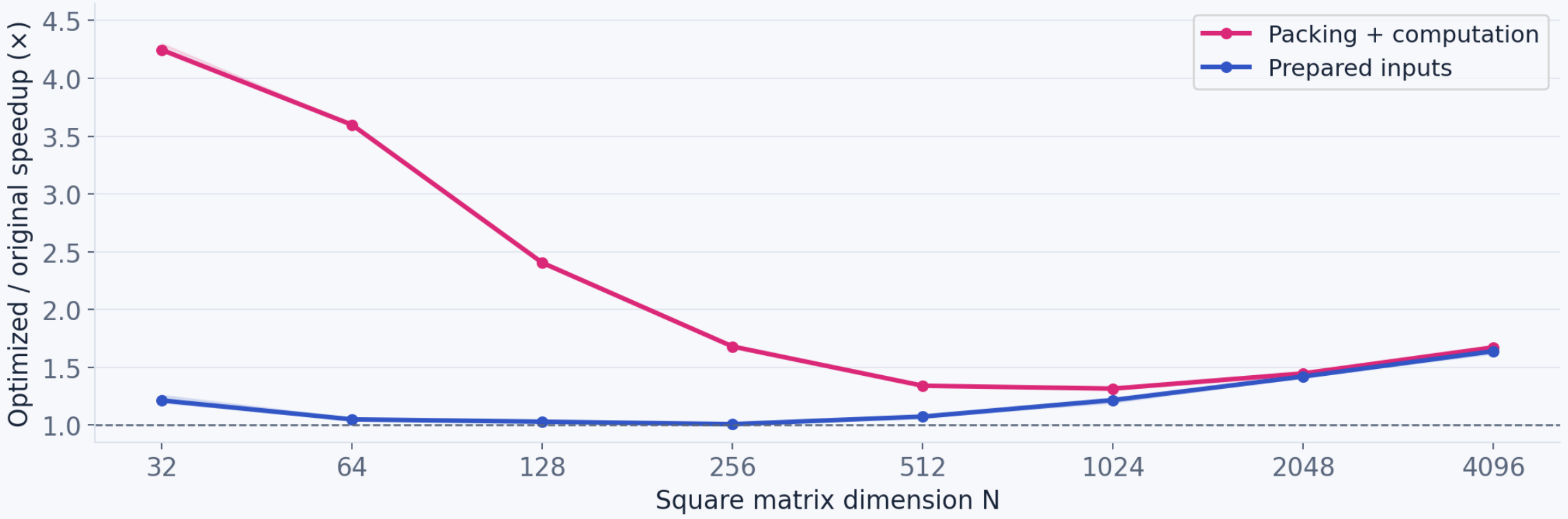
N = 2048

0.946×
≈6% longer time

Shading = IQR of paired-round speedups. It is not a confidence interval.

The optimized path improves preparation and compute

Same two SME backends; total and prepared-input timings answer different questions.



At N=4096: 1.67× total and 1.64× compute over original SME32.

Measured: examples/m4-pro/ · square FP32 · one calling thread · packing included unless stated

Read speedup together with absolute throughput

TOTAL mode · GFLOP/s from pooled median times · speedups from paired rounds

N	Original SME32	Optimized SME	Accelerate	Opt / original	Opt / Accelerate
128	640	**1539**	1407	2.40×	1.09×
512	1342	**1797**	1792	1.34×	1.00×
1024	1296	**1702**	1668	1.31×	1.02×
2048	1095	**1602**	1696	1.45×	0.95×
4096	945	**1576**	1571	1.67×	1.01×

No universal winner: performance depends on N, packing, cache behavior, and compiler output.

Measured: examples/m4-pro/ · square FP32 · one calling thread · packing included unless stated

Keep correctness, generated code, and timing together

The saved example is complete. A new run creates fresh data; slides never run the benchmark.

```
python3 build.py
python3 check.py
python3 benchmark.py --suite full --output results/run1
python3 plot.py results/run1
```

```
# Rebuild this deck from existing results only:
python3 slides/build_slides.py
```

Next experiments: independent runs, other sizes, controlled NEON baseline, and multicore scaling.

Scope of the evidence

Square FP32, one thread
Reused input buffers
One machine / compiler
No isolated ISA speedup
No per-change ablation

Fast GEMM comes from arithmetic, operand delivery, and measurement discipline together.

Sources: README.md · check.py · build.py · benchmark.py · plot.py · slides/build_slides.py

Follow the explanation back to its sources

Primary architecture/library references and the exact local implementation.

[1] Arm C Language Extensions

Streaming interfaces, ZA state, SME/SME2 intrinsics

[2] Hello SME / University Jena

FMOPA, streaming vector length, tile reuse

[3] Apple Accelerate

BLAS/LAPACK and runtime processor selection

[4] Apple BLAS threading

Single-thread execution control

Local reading order

src/sme32.c → original kernel
src/sme_pack.c → A transpose
src/pack_opt.c → dispatch + B packing
src/sme_opt.c → optimized kernels
examples/m4-pro/ → all measured data
speaker_notes.md → detailed explanations

Architecture diagrams are explanatory; every performance curve uses local measured results.